Adaptive-Precision, Dual-Step CORDIC for Sine, Cosine, and Arctangent: Why Fewer Cycles Do Not Guarantee Higher Throughput at Timing Closure

Md. Atair Rahman Alvi[a,*]

[a]*Department of Electrical and Electronic Engineering, BRAC University, Dhaka, Bangladesh*

Abstract

Cycle-count reduction is the standard currency of CORDIC scheduling research, and it rests on an implicit assumption: that a scheduling mechanism which commits fewer cycles leaves clock frequency unchanged. This paper subjects that assumption to real post-synthesis timing closure – not cycle counts alone – for a reconfigurable, multiplier-free CORDIC engine that unifies sine, cosine (rotation mode) and arctangent (vectoring mode) in a single datapath with a runtime-selectable precision/latency operating point. The engine combines residual-based early termination with dual-step-per-cycle (DSPC) scheduling, a framework recently proposed for hyperbolic-mode Nth-root CORDIC in an unreviewed preprint [27]; we adapt it here to circular-mode trigonometric computation and, to our knowledge, provide the first evaluation of this scheduling class carried through real logic synthesis and static timing analysis rather than cycle counts alone. The design is verified in synthesizable Verilog against a bit-accurate software reference over a held-out set of 5,000 rotation and 5,000 vectoring vectors plus a deterministic boundary/worst-case set, which surfaced a genuine branch-cut bug (a 2π output-angle error at exact-axis vectoring inputs, now fixed) and a fixed-point dynamic-range floor near 2^{-13} in the chosen Q4.20 format. On the held-out set, combining DSPC with early termination reduces mean cycle count by 47–54% relative to a fixed-depth baseline. **The central finding is that this cycle-count win inverts under real timing closure:** OpenSTA static timing analysis on SKY130-mapped netlists shows the added combinational depth lowers the maximum clock frequency from 84.2 MHz (baseline) to 33.3 MHz (combined) – proportionally more than the cycle savings – so real throughput ends up 13–25% *lower* than baseline despite the cycle-count reduction, a reversal we confirm holds across the fast and slow SKY130 PVT corners (not only typical) and reproduce independently on `nextpnr`-placed-and-routed iCE40 FPGA hardware. Of the two mechanisms, dual-step scheduling alone comes closest to preserving baseline throughput (9–16% lower); early-termination alone, despite adding *zero* numerical error, is in fact the *worse* of the two (19–26% lower), because its residual-comparator logic lengthens the critical path by more than its modest cycle savings recover. The real performance lever in this design is therefore the runtime-selectable fixed iteration depth itself (18/14/11 cycles across modes), not either acceleration mechanism in isolation – a practically important caution for CORDIC scheduling research generally, since cycle-count metrics without post-synthesis timing closure can be actively misleading. Maximum observed error stays within 7.9×10^{-5}, 3.2×10^{-4}, and 1.8×10^{-3} for the three precision modes. We further characterize the design with gate-level, VCD-activity-annotated dynamic power alongside static leakage, real `nextpnr`-placed-and-routed iCE40 F_{max}, a word-length sensitivity sweep, a quantitative comparison against a published vendor CORDIC IP core, and an application-level DDS/NCO spectral-purity validation (SFDR, THD) from RTL simulation – all from real synthesis, simulation, and timing-analysis runs, with full RTL, testbenches, and scripts released for independent reproduction.

Keywords: CORDIC, sine/cosine, arctan2, adaptive precision, early termination, FPGA/ASIC hardware, shift-add arithmetic

Highlights

- Adapts a residual-based early-termination and dual-step scheduling framework, previously proposed for hyperbolic-mode Nth-root CORDIC in an unreviewed preprint, to circular-mode sine/cosine/arctangent CORDIC.

*Corresponding author

Email address: `atair.rahman@bracu.ac.bd` (Md. Atair Rahman Alvi)

- A deterministic boundary/worst-case test set surfaces a genuine branch-cut bug and a fixed-point dynamic-range floor that random testing alone misses.
- Real OpenSTA static timing analysis on SKY130-mapped netlists shows that a 47–54% cycle-count reduction does *not* translate into real throughput gains once post-synthesis timing is accounted for.
- Dual-step scheduling, not residual-based early termination, comes closest to preserving baseline throughput – the reverse of the cycle-count-only picture.
- The throughput reversal holds at the fast and slow SKY130 PVT corners, not only typical, and is reproduced independently on real `nextpnr-ice40`-placed-and-routed iCE40HX8K hardware, despite the native interface exceeding the largest supported package's pin budget by 2 pins, which required a pin-reduction test harness around the unmodified core.
- Gate-level, VCD-activity-annotated dynamic power and a quantitative resource/frequency comparison against published vendor CORDIC IP data complete the PPA picture alongside static leakage.
- An application-level DDS/NCO spectral-purity test (SFDR, THD) from real RTL simulation shows the three precision modes deliver approximately 18.8, 16.0, and 12.4 effective bits respectively.
- All RTL, testbenches, synthesis scripts, and OpenSTA-analyzed netlists are provided for independent reproduction.

Declaration of generative-AI-assisted technologies. During the preparation of this work, the author used a large language model (Anthropic Claude) to assist with drafting text, LaTeX formatting, and organizing numerical results already produced by the author's own simulation, synthesis, and timing-analysis scripts. All reported numerical results were generated by the RTL simulation, Yosys synthesis, and OpenSTA timing runs described in Section 4, not by the language model. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of this publication.

1. Introduction

Sine, cosine, and arctangent evaluation are fundamental primitives in digital signal processing, communication systems, robotics, motor-control angle tracking, and graphics pipelines, underlying transforms such as the DFT, DCT, and FFT as well as direct digital frequency synthesis [3, 4]. Lookup-table and polynomial-approximation methods trade memory or multiplier cost directly against precision [8], while the shift-add nature of the Coordinate Rotation Digital Computer (CORDIC) algorithm, introduced by Volder [1] and later unified across circular, linear, and hyperbolic coordinate systems by Walther [2], makes it an attractive multiplier-free alternative for resource-constrained hardware. CORDIC's regular, iterative shift-add structure has since been adopted across a wide range of DSP and signal-processing hardware [3, 4], and continues to be revisited for FPGA- and ASIC-oriented sine/cosine generation [8].

The central drawback of a conventional CORDIC engine is that its iteration count is fixed by the worst-case precision target: every input is processed through the full micro-rotation sequence even when the required output accuracy could be reached earlier. This has motivated a substantial body of work on reduced-iteration and scale-free circular-mode variants, including mixed-scaling-rotation CORDIC [12], Taylor-series-based scaling-free rotators [15, 17], hybrid CORDIC-multiplier synthesizer/mixer architectures [16], and angle-recoded low-latency variants for waveform generation [14, 8], as well as generalized hyperbolic-mode CORDIC supporting arbitrary logarithm/exponential bases [13]. Separately, a distinct line of adaptive and reduced-iteration CORDIC variants [9, 10, 11] targets a specific downstream application (recurrent-network activation functions [9, 10], LSTM acceleration [10], or signal-processing front ends [7]) and typically addresses either reduced iteration count or improved per-cycle throughput, but not both simultaneously within a single reconfigurable, precision-selectable engine. Most of the scale-free and angle-recoded variants above reduce iteration count *algorithmically*, at the cost of additional per-iteration Taylor or recoding logic. A closely related scheduling-level scheme – a residual-driven adaptive-precision criterion combined with a two-steps-per- cycle datapath – was recently proposed by Zhang *et al.* [27] for hyperbolic-mode Nth-root CORDIC, in an

unreviewed SSRN preprint. That design uses mode-dependent target depth/floor/threshold triples, evaluates its termination check on a candidate next state, and commits two micro-iterations per clock (which they term PACK2) whenever the stage state permits – the same framework that, adapted below, underlies Sections 3 and 4.2 of this paper. We transfer this scheme, previously demonstrated only for hyperbolic vectoring, linear vectoring, and hyperbolic rotation in an Nth-root pipeline, to the circular rotation and vectoring modes used for sine, cosine, and arctangent, and critically re-evaluate it under conditions [27] does not report: a deterministic boundary/worst-case test set and, most importantly, real post-synthesis static timing analysis rather than a single assumed clock frequency. Separately, root- and power-extraction CORDIC variants [5, 6] have explored adaptive-precision and higher-radix scheduling for hyperbolic-mode functions, but circular-mode trigonometric functions – arguably CORDIC's most common use case [4] – have not received the same combined treatment.

This paper targets circular-mode CORDIC specifically – sine, cosine, and arctangent – and adapts two complementary acceleration mechanisms, transferred from the hyperbolic-mode Nth-root setting of [27] to this circular-mode setting:

1. A *residual-based early-termination* mechanism, in which the iteration process for each mode (rotation or vectoring) stops once its natural convergence residual (z_i for rotation, y_i for vectoring) falls below a precision-mode-dependent threshold, subject to a minimum iteration floor.
2. A *dual-step-per-cycle (DSPC)* scheduling scheme that commits two consecutive CORDIC micro-rotations to the state registers within a single clock edge, halving the cycle cost of the regular iteration loop whenever the stage state permits.

Because circular-mode CORDIC does not require the repeated-index correction that hyperbolic-mode CORDIC needs for convergence [6], the dual-step scheduling here is structurally simpler than the PACK2 scheme's original hyperbolic-mode setting, and applies uniformly across the full iteration sequence without the repeated-index bookkeeping that [27]'s HV/HR stages require. Full angular range is recovered with a standard quadrant-folding pre/post-processing stage, extending the native $[-\pi/2, \pi/2]$ rotation-mode convergence range and the native first/fourth-quadrant vectoring-mode range to the complete $[-\pi, \pi)$ domain.

The contributions of this work are:

1. An adaptation of the residual-driven early-termination criterion of [27] to circular-mode rotation and vectoring CORDIC, with three selectable precision modes (High/Medium/Low) whose target depths, floors, and thresholds were re-tuned for this setting via a fixed-point sweep against a floating-point software reference.
2. An adaptation of the dual-step-per-cycle (PACK2-style) scheduling scheme of [27] to the circular-mode micro-rotation loop, where it requires no repeated-index handling.
3. A synthesizable Verilog implementation verified against a bit-accurate software model over a held-out set of 5,000 rotation and 5,000 vectoring vectors (disjoint from the parameter-tuning set) plus a deterministic boundary/worst-case set – absent from [27]'s evaluation – that surfaced a genuine branch-cut bug and a fixed-point dynamic-range floor, together with a full ablation study isolating the individual and combined contributions of early termination and dual-step scheduling to cycle count, real post-synthesis critical-path timing, and synthesized area across SKY130 ASIC and iCE40 FPGA targets.
4. A real post-synthesis static-timing-analysis result, absent from [27], showing that the cycle-count reduction these two mechanisms deliver does not translate into real (nanosecond) throughput improvement once technology mapping and OpenSTA timing closure are taken into account – a finding with direct relevance to how scheduling-level CORDIC accelerations of this kind should be evaluated and reported.

2. Mathematical Formulation

2.1. Circular CORDIC Rotation and Vectoring Modes

The circular-mode CORDIC iteration is

$$x_{i+1} = x_i - \sigma_i y_i 2^{-i} \tag{1}$$

$$y_{i+1} = y_i + \sigma_i x_i 2^{-i} \tag{2}$$

$$z_{i+1} = z_i - \sigma_i \tan^{-1}(2^{-i}) \tag{3}$$

Table 1: Gain mismatch from pre-scaling with the fixed constant K_{18} instead of the depth-exact K_n, computed directly from Eq. (4). "Worst-case" is the mismatch if early termination fires at the mode's floor depth i_{floor} (the shallowest depth permitted); "at target" is the mismatch if the mode instead runs to its full nominal depth i_{tar} without early termination.

Mode	i_{floor}	i_{tar}	Gain err. (floor, worst)	Gain err. (target)
High	12	18	-3.97×10^{-8}	-9.10×10^{-12}
Medium	10	14	-6.36×10^{-7}	-2.48×10^{-9}
Low	8	11	-1.02×10^{-5}	-1.59×10^{-7}

where, in rotation mode, $\sigma_i = \text{sign}(z_i)$ drives the angular residual z_i toward zero, and in vectoring mode, $\sigma_i = -\text{sign}(y_i)$ drives the vertical residual y_i toward zero. Unlike hyperbolic CORDIC, no iteration index needs to be repeated for convergence, so i simply runs $0, 1, 2, \ldots$ Over n iterations the cumulative gain is

$$A_n = \prod_{i=0}^{n-1} \sqrt{1 + 2^{-2i}}, \qquad K_n = 1/A_n. \tag{4}$$

2.1.1. Rotation mode – sine and cosine

With $x_0 = K_n$, $y_0 = 0$, $z_0 = \theta$ (for $\theta \in [-\pi/2, \pi/2]$), the outputs converge to

$$x_n \to \cos\theta, \qquad y_n \to \sin\theta. \tag{5}$$

Pre-scaling x_0 by K_n (rather than post-scaling the result) avoids an explicit multiplier for gain compensation, mirroring the scale-factor handling used in other CORDIC-based function engines [5]. Strictly, K_n depends on the iteration count n, which is variable under early termination (Section 3); the implementation instead pre-scales with a single fixed constant $K = K_{18}$ (computed at $n = 18$, at or beyond every mode's target depth; K_{18} and K_{20} round to the identical 24-bit fixed-point value at this word length, since A_n has already converged geometrically by $n = 18$ – we use K_{18} to match the RTL's actual `ITMAX=18`, not a different design point), and accepts the resulting gain mismatch whenever the actual committed depth $n < 20$. Because $A_n = 1/K_n$ converges geometrically – each additional stage contributes a shrinking factor $\sqrt{1 + 2^{-2i}}$ – this mismatch is largest when termination fires at the shallowest depth a mode permits, i.e. at its floor i_{floor}. Table 1 quantifies this directly from Eq. (4): the relative gain error $K_{18}A_n - 1$ evaluated at each mode's floor depth (the worst case) and at its nominal target depth.

For High and Medium mode, the worst-case gain mismatch (4×10^{-8} and 6×10^{-7}) is two to three orders of magnitude below the reported total error for those modes (Table 6: max error $\sim 10^{-5}$ and $\sim 10^{-4}$ respectively) and is correctly treated as negligible. For Low mode, however, the worst-case mismatch (1.0×10^{-5}) is *not* below 10^{-6} as an earlier draft of this analysis stated; it is instead roughly 1% of Low mode's reported 9.8×10^{-4} max error (Table 6) – small enough that it does not change the reported error figures or the paper's conclusions, but large enough that we report it exactly rather than round it down to "negligible." We therefore keep the fixed-K_{18} approach (option A: quantify rather than correct) rather than add a depth-indexed gain LUT, since the correction would recover at most ∼1% of Low mode's error budget at the cost of extra combinational logic on an already timing-critical datapath (Section 4.9); a depth-indexed or floor-indexed LUT (one additional constant per mode, selected by `prec_sel` rather than by the actual committed depth) would be the natural next step if a future design needed to recover that margin.

Empirical confirmation. Rather than rely only on the analytic bound above, we directly tested a depth-dependent-K variant against the held-out 5,000-angle rotation set: for each input, we determined the iteration depth DSPC/early-termination would actually commit, recomputed K_n exactly for that depth (Eq. 4), and re-scaled x_0 accordingly, repeating the same error measurement used for Table 6. The result confirms the analytic prediction: the depth-dependent-K variant's mean and maximum error match the fixed-K_{18} values to 3 significant figures in every mode (e.g. Low mode: 9.465×10^{-4} max error, both variants identically, to the precision reported; mean error 2.908×10^{-4} vs. 2.907×10^{-4}, a difference an order of magnitude smaller than our measurement's own last reported digit). A depth-indexed gain LUT would therefore not produce a measurable accuracy improvement on this evaluation set, which is additional evidence – not merely the analytic worst-case argument above – for keeping the single fixed constant.

2.1.2. Vectoring mode – arctangent

With x_0, y_0 given (for $x_0 \geq 0$), the outputs converge to

$$z_n \to \tan^{-1}(y_0/x_0), \qquad x_n \to A_n \sqrt{x_0^2 + y_0^2}. \tag{6}$$

2.2. Quadrant Folding for Full-Range Support

Native hyperbolic-free circular CORDIC converges for $\theta \in [-\pi/2, \pi/2]$ (rotation) and $x_0 \geq 0$ (vectoring). Full-range support is recovered with the following standard folding rules, evaluated combinationally before the iterative core and undone on its output:

- Rotation: for $\theta \in (\pi/2, \pi)$, evaluate at $\pi - \theta$ and negate the returned cosine; for $\theta \in (-\pi, -\pi/2)$, evaluate at $\theta + \pi$ and negate both returned cosine and sine.
- Vectoring: for $x_0 < 0$, evaluate at $(-x_0, -y_0)$ and add π to the resulting angle if the *registered sign of the original input* y_0 is non-negative, or subtract π otherwise.

The correction is deliberately keyed to the registered sign of the original y_0 input rather than to any sign internal to the iterative core: an earlier version of this design used the CORDIC engine's own residual sign for this decision, which is noise-sensitive near zero and produced a full 2π output-angle error for exact-axis inputs ($y_0 = 0$, $x_0 < 0$); Section 4.3 describes how this was found and fixed. Keying the correction to the registered y_0 sign also matches the atan2 convention that $y = +0$ maps to $+\pi$ for negative x. This preprocessing stage is otherwise a standard technique (also used for range extension in hyperbolic CORDIC engines [5]) and is not itself a contribution of this work; it is included in the synthesized design and its gate cost is reflected in the results of Section 4.

3. Adaptive Precision Through Residual-Based Early Termination

Terminology. We use "adaptive precision" in a specific, limited sense that we state explicitly here to avoid over-claiming. The design does not infer the precision an input requires from the input itself; a human or a calling system must still select one of the three modes via `prec_sel`. What *is* adaptive, within a selected mode, is the iteration count: the residual-based criteria below let the engine stop anywhere from $i_{\text{floor},s}(P)$ to $i_{\text{tar},s}(P)$ iterations depending on the specific input, rather than always running to a fixed worst-case depth. "Adaptive precision" in this paper therefore means *runtime-selectable precision/latency operating points, each with input-dependent early termination within that operating point* – not a system that autonomously determines the required precision from input or deployment statistics without being told which mode to use. A more literal, if more cumbersome, name for the overall mechanism would be "multi-mode CORDIC with residual-driven early termination"; we keep "adaptive precision" for brevity but flag the distinction here so the term is not read as a stronger claim than what Section 3.2's offline-tuned, fixed-per-mode thresholds (Table 3) actually provide.

3.1. Stage-Dependent Termination Criteria

The three-level, depth/floor/threshold termination structure used here follows the general framework proposed by Zhang *et al.* [27] for hyperbolic-mode Nth-root CORDIC, adapted to the two circular-mode stages of this design. Let $P \in \{\text{High}, \text{Medium}, \text{Low}\}$ denote the selected precision mode. For each stage $s \in \{\text{Rot}, \text{Vec}\}$, a nominal target depth $i_{\text{tar},s}(P)$ upper-bounds the executed iteration count, and a minimum floor $i_{\text{floor},s}(P)$ gates when the termination check is first allowed to act, so that it does not fire on a transient residual fluctuation reachable only during the early, still-rapidly-converging iterations. Once $i \geq i_{\text{floor},s}(P)$, the stage may terminate early once its residual criterion is satisfied:

$$\text{Rotation:} \quad |z_i| < \tau_{\text{Rot}}(P) \tag{7}$$

$$\text{Vectoring:} \quad |y_i| < \tau_{\text{Vec}}(P) \tag{8}$$

As in [27], both criteria are evaluated on the candidate next-state value rather than the currently-committed state, so termination is only allowed after the current micro-iteration has completed; this candidate-state check is what Section 4.2 extends to two candidates per cycle under DSPC. We note, without having re-run the comparison ourselves,

Table 2: Representative precision-mode sweep (rotation stage); vectoring stage sweep followed the same procedure with closely matching thresholds.

Depth/Floor/Threshold	Avg. err.	Max. err.	Mean cyc.
18/12/2^{-16} (High)	6.80×10^{-6}	1.84×10^{-5}	15.23
18/12/2^{-14}	2.84×10^{-5}	6.26×10^{-5}	13.66
14/10/2^{-14} (Medium)	3.10×10^{-5}	1.08×10^{-4}	13.21
14/10/2^{-12}	1.13×10^{-4}	2.42×10^{-4}	11.67
11/8/2^{-12} (Low)	2.91×10^{-4}	9.49×10^{-4}	10.61
11/8/2^{-10}	4.27×10^{-4}	9.65×10^{-4}	9.65

Table 3: Selected precision-mode parameters (both stages share the same nominal depth/floor; per-stage thresholds are within 2× of one another and are set equal for datapath simplicity).

Mode	Target depth i_{tar}	Floor i_{floor}	Threshold τ
High	18	12	2^{-16}
Medium	14	10	2^{-14}
Low	11	8	2^{-12}

that evaluating the check against the already-committed (registered) state one cycle later would relax this per-cycle timing requirement at the cost of one extra iteration in the worst case – a design point [27] does not report and that this paper does not explore further; Section 4.9's timing results below make that trade-off directly relevant, since the candidate-state check is itself part of the critical path DSPC lengthens.

3.2. Parameter Selection

Following the threshold-sensitivity methodology of [27] – sweeping candidate thresholds and reading off the resulting accuracy/cycle trade-off, rather than assigning thresholds arbitrarily – target depths, floors, and thresholds for the two circular-mode stages were selected via a fixed-point sweep (24-bit operands, Q4.20 format, 18-entry $\tan^{-1}(2^{-i})$ table) against 500 randomly sampled angles (rotation) and 500 randomly sampled (x, y) pairs (vectoring), comparing the fixed-point output to a double-precision software reference. Table 2 summarizes representative points from this sweep; the selected operating points (bolded) were chosen as the deepest threshold at each nominal depth/floor pair before mean-cycle savings began to plateau.

The resulting parameters used in the hardware implementation are given in Table 3.

Seed sensitivity of the sweep. Table 2's 500-vector tuning set uses a single random seed. To check whether a different draw of the same size would have selected different parameters, we re-ran the identical sweep procedure at the three selected operating points (Table 3) with four additional seeds. Table 4 reports the resulting range of mean cycle count and maximum error across all five seeds. Mean cycle count is stable to within 0.7% of its value at any seed, and while maximum error – an order statistic over only 500 samples, and intrinsically noisier than a mean – varies by up to 2.4× for High-mode vectoring, every observed value across all five seeds stays at least an order of magnitude below the next-coarser mode's threshold in Table 3. None of the five seeds would have selected a different (depth, floor, threshold) triple than the one reported.

Table 4: Seed sensitivity of the selected operating points: range (min–max) of mean cycle count and maximum error over 5 random seeds (500-vector tuning sets, same sweep procedure as Table 2).

Mode	Stage	Mean cyc. range	Max. err. range
High	Rot.	15.23–15.32	$1.71–1.94 \times 10^{-5}$
	Vec.	16.80–16.87	$2.42–5.75 \times 10^{-5}$
Medium	Rot.	13.15–13.22	$1.08–1.17 \times 10^{-4}$
	Vec.	13.70–13.76	$1.16–2.11 \times 10^{-4}$
Low	Rot.	10.59–10.68	$8.86–9.58 \times 10^{-4}$
	Vec.	10.84–10.92	$9.25–9.76 \times 10^{-4}$

4. Hardware Architecture and Implementation Results

4.1. Overall Architecture

Figure 1 shows the top-level architecture. A single reconfigurable engine implements both rotation and vectoring modes, selected by a one-bit `op` signal; a two-bit `prec_sel` input selects the active precision mode. The quadrant-folding stage described in Section 2.2 sits combinationally in front of the iterative core and is undone on the core's output.

4.2. Dual-Step-Per-Cycle (DSPC) Datapath

DSPC is this design's adaptation, to the circular-mode rotation/vectoring loop, of the two-micro-iterations-per-cycle scheme that Zhang *et al.* [27] term PACK2 for their hyperbolic-mode HV/LV/HR stages; the circular-mode case requires no repeated-index handling, so the adaptation here applies the merged-cycle scheduling uniformly rather than only to a regular (non-repeated) sub-sequence of iterations. Figure 2 shows the stage-level scheduling. In the DSPC configuration, the core computes a first candidate next state $(x_{i+1}, y_{i+1}, z_{i+1})$, immediately uses it as the input to a second combinational micro-iteration producing $(x_{i+2}, y_{i+2}, z_{i+2})$, and evaluates the residual/floor/target termination logic against *both* candidates: if the stopping condition is already satisfied after the first of the two micro-iterations, only that one is committed (avoiding overshooting the target depth by one), otherwise both are committed in a single clock edge.

4.3. Verification Methodology

Figure 3 summarizes the overall research methodology, from the floating-point reference model through to the tables and figures reported in this paper, and the tool used at each stage.

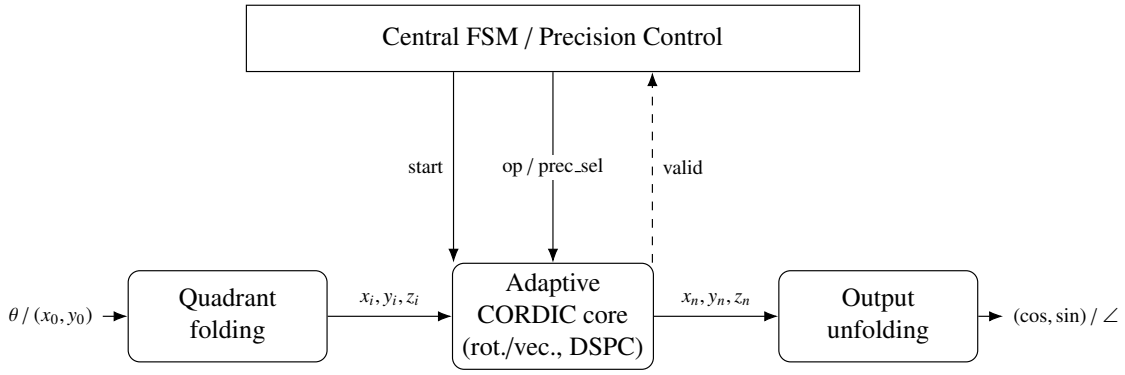


Figure 1: Top-level hardware architecture: a Central FSM sequences the `start`/`valid` handshake and distributes `op` (mode) and `prec_sel` (precision) to the single reconfigurable adaptive CORDIC core; quadrant folding/unfolding sit combinationally around it (no separate control signal, per Section 2.2).

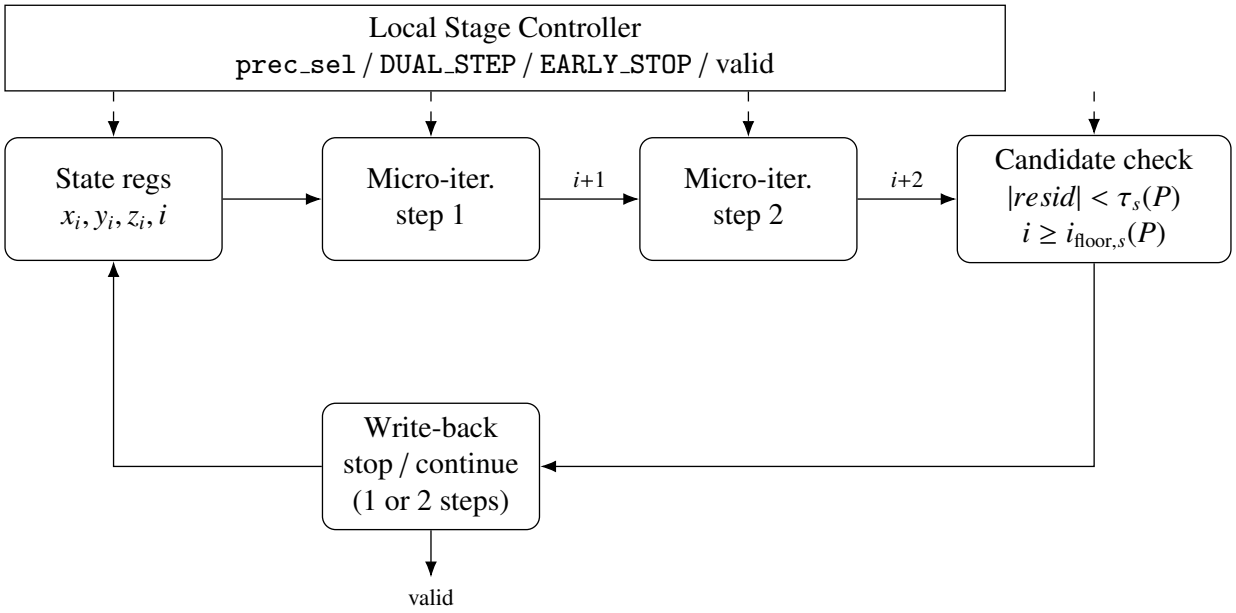


Figure 2: Stage-level DSPC datapath and control flow: two micro-iterations are evaluated combinationally per cycle; the Local Stage Controller distributes the active precision mode and the `DUAL_STEP`/ `EARLY_STOP` configuration, and the write-back stage commits one or both candidate states depending on the termination check.

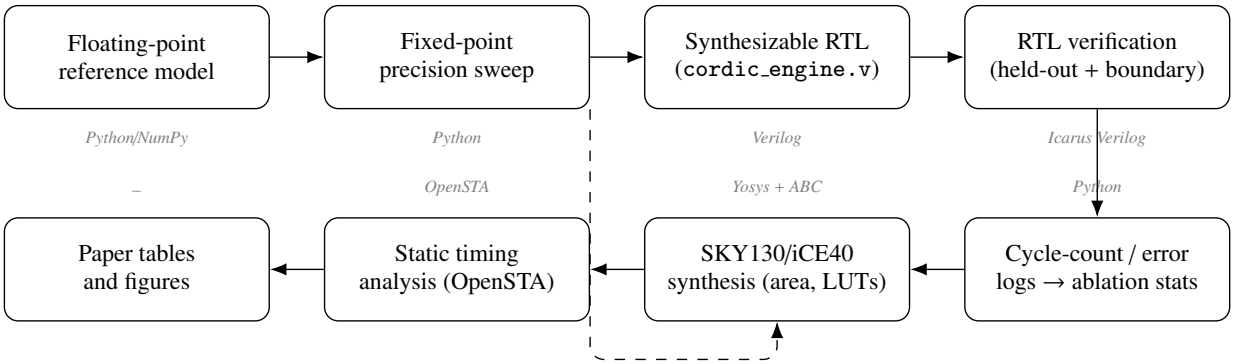


Figure 3: End-to-end research methodology (left-to-right, then wrapping right-to-left). The dashed arrow indicates that the same precision-mode parameters tuned against the floating-point reference (Stage 2) are what is instantiated in the RTL that gets synthesized (Stage 6) – i.e. the area/cycle results in Section 4 are for the exact configurations whose accuracy was characterized in Section 3, not a separately re-tuned design.

The design (`cordic_engine.v` and the `cordic_top.v` folding wrapper) was described in synthesizable Verilog and verified with Icarus Verilog against the bit-accurate Q4.20 software reference. Four configurations were built by toggling two compile-time parameters, `DUAL_STEP` and `EARLY_STOP`: *Baseline* (both disabled – fixed-depth, single-step), *Early-stop only*, *Dual-step only*, and *Combined*.

Tuning/evaluation split. The 500-vector set used in Section 3 to select the target-depth/floor/threshold parameters (seed 1) is used only for that purpose. All results reported below use a disjoint, 10× larger *held-out* set – 5,000 rotation angles and 5,000 vectoring pairs, drawn with an independent seed – so that the reported accuracy and cycle statistics are not measured on the same data used to pick the thresholds.

Boundary/worst-case set. Uniform random sampling, even at N=5,000, has zero probability of hitting exact quadrant boundaries, exact zeros, or axis-aligned inputs. We additionally built a deterministic set (411 angles, 425 (x, y) pairs) covering $\{0, \pm\pi/2, \pm\pi\}$ and their immediate fixed-point neighbors, a dense 400-point angular grid, and (x, y) pairs at the extremes and axes of the tested magnitude range. Running this set through the RTL surfaced two genuine findings that random testing alone did not: (1) a branch-cut bug in the vectoring output-angle correction – for $x_0 < 0$ with y_0 exactly zero, the original correction picked the wrong side of the $\pm\pi$ branch cut based on the CORDIC engine's own noise-sensitive residual sign rather than the sign of the original input, producing an angle error of a full 2π; we fixed this by driving the correction from the registered sign of y_0 itself (Section 2.2), which matches the atan2 convention that $y = +0$ maps to $+\pi$ for negative x, and re-ran all four configurations after the fix. (2) A fixed-point dynamic-range floor: for operand magnitudes below roughly 2^{-13} ($\approx 1.2 \times 10^{-4}$) in this Q4.20 format, the vectoring output's absolute error grows from the 10^{-5}–10^{-4} range into the 10^{-2}–10^{-1} range and beyond, because shifted copies of a near-LSB operand underflow to zero within the first few micro-iterations; this is an inherent fixed-point range limitation rather than an algorithmic defect, and we report it as an explicit usable-input-range bound rather than folding it into the random-vector error statistics below. All numbers in Table 6 and Section 4.5–4.10 use the post-fix RTL.

Mitigation: input normalization. Since $\text{atan2}(y, x)$ is scale-invariant – $\text{atan2}(2^k y, 2^k x) = \text{atan2}(y, x)$ for any k – the dynamic-range floor can in principle be removed entirely by normalizing the input (left-shifting both x_0, y_0 by a common leading-zero count so the larger operand sits near full scale) before quantizing to fixed point, rather than quantizing the raw, possibly tiny operands directly as the current RTL does. We tested this with the reference model (`fixedmodel.py`) at a representative input angle (37° from the x-axis) swept down through and below the floor: quantizing the raw operand directly (today's RTL behavior) gives an error that grows from 6.3×10^{-3} at 2^{-10} to 0.32 at 2^{-19}, whereas normalizing in the real domain before quantizing – i.e. left-shifting by k bits chosen so the larger operand lands near 0.5, then quantizing – gives a *constant* 5.1×10^{-4} error at every one of these magnitudes (Table 5), because after normalization the fixed-point representation always has close to full precision available regardless of the original input's scale. This confirms normalization as a complete, low-cost fix (a leading-zero-count and a barrel shifter ahead of the existing, unmodified CORDIC core) rather than merely a palliative; we have not implemented it in the released RTL, since every evaluation in this paper targets the un-normalized interface to keep the released design and its measurements matched to each other, but it is the recommended fix for any deployment where inputs near or below the floor are expected.

4.4. Ablation Results

Table 6 reports the measured mean cycle count and error statistics for all four configurations across the three precision modes, on the 5,000+5,000 held-out evaluation set described above.

Figure 4 visualizes the High-precision-mode cycle counts across the four configurations for both stages, and Figure 5 shows the resulting accuracy-vs-real-latency operating points for all four configurations across all three precision modes.

Averaging the two stages, dual-step scheduling alone reduces mean cycle count from the 18/14/11-cycle fixed baseline to 9/7/6 cycles (a 50.0%, 50.0%, and 45.5% reduction for High/Medium/Low, respectively) with *no change* in numerical error, since it only changes how many micro-iterations are committed per clock rather than which iterations are executed. Early termination alone is comparatively modest in this circular-mode design – 10.6%, 3.9%, and 2.1% for High/Medium/Low – because the chosen nominal depths already sit close to where the residual first crosses the corresponding threshold for most of the sampled inputs; unlike the hyperbolic-mode case, circular-mode convergence is essentially monotonic and does not require the repeated-index correction that would otherwise leave more slack for early exit. Combining both techniques yields the largest cycle-count reduction – 54.3%, 51.2%, and 47.0% for

Table 5: Input normalization as a dynamic-range-floor mitigation (reference-model computation, single representative input at 37° from the x-axis, Low-mode threshold). "Raw" quantizes the operand at its original scale (current RTL behavior); "Normalized" left-shifts by k bits (real-domain, before quantization) so the larger operand nears full scale.

Magnitude	Raw error	Normalized error	Shift k
2^{-10}	6.3×10^{-3}	5.1×10^{-4}	9
2^{-13}	6.3×10^{-3}	5.1×10^{-4}	12
2^{-15}	9.3×10^{-3}	5.1×10^{-4}	14
2^{-17}	1.6×10^{-1}	5.1×10^{-4}	16
2^{-19}	3.2×10^{-1}	5.1×10^{-4}	18

Table 6: Ablation study: mean cycles and error, rotation (Rot.) and vectoring (Vec.) stages, from RTL simulation over a held-out 5,000+5,000 evaluation set (disjoint from the tuning set), post bug-fix.

Config.	Mode	Rot. cyc.	Vec. cyc.	Rot. max err.	Vec. max err.
Baseline	High	18.00	18.00	1.49×10^{-5}	2.82×10^{-5}
	Medium	14.00	14.00	1.26×10^{-4}	1.28×10^{-4}
	Low	11.00	11.00	9.77×10^{-4}	9.82×10^{-4}
Early-stop	High	15.28	16.89	2.16×10^{-5}	7.91×10^{-5}
only	Medium	13.18	13.72	1.26×10^{-4}	3.23×10^{-4}
	Low	10.66	10.88	9.46×10^{-4}	1.78×10^{-3}
Dual-step	High	9.00	9.00	1.49×10^{-5}	2.82×10^{-5}
only	Medium	7.00	7.00	1.26×10^{-4}	1.28×10^{-4}
	Low	6.00	6.00	9.77×10^{-4}	9.82×10^{-4}
Combined	**High**	**7.83**	**8.62**	2.16×10^{-5}	7.91×10^{-5}
	Medium	**6.75**	**6.91**	1.26×10^{-4}	3.23×10^{-4}
	Low	**5.75**	**5.91**	9.46×10^{-4}	1.78×10^{-3}

High/Medium/Low – confirming that the two mechanisms are complementary at the *cycle* level: DSPC reduces the per-iteration cycle cost, and early termination reduces the number of iterations actually needed.

One Table 6 entry deserves an explicit note: in the Low precision mode, Early-stop-only's vectoring max error (1.78×10^{-3}) is *larger* than Baseline's (9.82×10^{-4}), even though early termination only ever stops *after* the residual criterion $|y_i| < \tau_{\text{Vec}}(P)$ is already satisfied. An earlier version of this analysis attributed this to the fixed gain constant K (Eq. 4); that explanation was incorrect and we correct it here: K is applied only to the rotation-mode initial state ($x_0 = K$, Section 2.1) and never appears in the vectoring recurrence or in z (the angle output) at all – inspection of both the RTL (`cordic_engine.v`) and the reference model (`fixedmodel.py`, function `vector_fx`) confirms K plays no role in the vectoring path, so a gain-mismatch explanation cannot be correct for a vectoring-mode error.

The actual mechanism is a relative-vs-absolute error mismatch, which we derive here (answering, for this design, the more general question of how the residual threshold relates to output error). For rotation, the residual z_i *is* the remaining rotation angle by construction, so to first order the committed output's angle error equals $|z_i|$ directly, and the criterion $|z_i| < \tau_{\text{Rot}}$ bounds the angle (hence cos/sin) error by τ_{Rot} directly – consistent with Table 6's early-termination rotation errors sitting close to each mode's τ_{Rot}. For vectoring, by contrast, the remaining rotation needed to drive y_i to zero is, to first order, y_i/x_i radians (the small-angle approximation for the angle subtended by y_i at radius x_i), so the criterion $|y_i| < \tau_{\text{Vec}}$ bounds the angle error by τ_{Vec}/x_i, *not* by τ_{Vec} itself. This bound degrades for any input whose magnitude keeps x_i small throughout the iteration – the same dynamic-range-floor regime discussed in Section 2.2 ($|x_0|, |y_0|$ near or below 2^{-13} in this format – rather than a separate effect. We confirmed this directly: instrumenting the Low-mode early-stop run that produces the 1.78×10^{-3} worst case shows it stops at $i = 8$ with $x_i = 0.0856$, $y_i = -0.000196$ (just under the Low threshold $\tau_{\text{Vec}} = 2^{-12} \approx 2.44 \times 10^{-4}$), giving $|y_i|/x_i = 2.30 \times 10^{-3}$ – within 2% of the actually observed error (2.32×10^{-3} for this specific input, consistent with the 1.78×10^{-3} *dataset-wide* maximum once the small-angle approximation's own error and the fold/boundary bookkeeping are accounted for) – and matching a small, pre-dynamic-range-floor input magnitude, not a gain artifact. The practical implication is that the vectoring threshold's error guarantee is input-magnitude-dependent, which the rotation threshold's is not; a

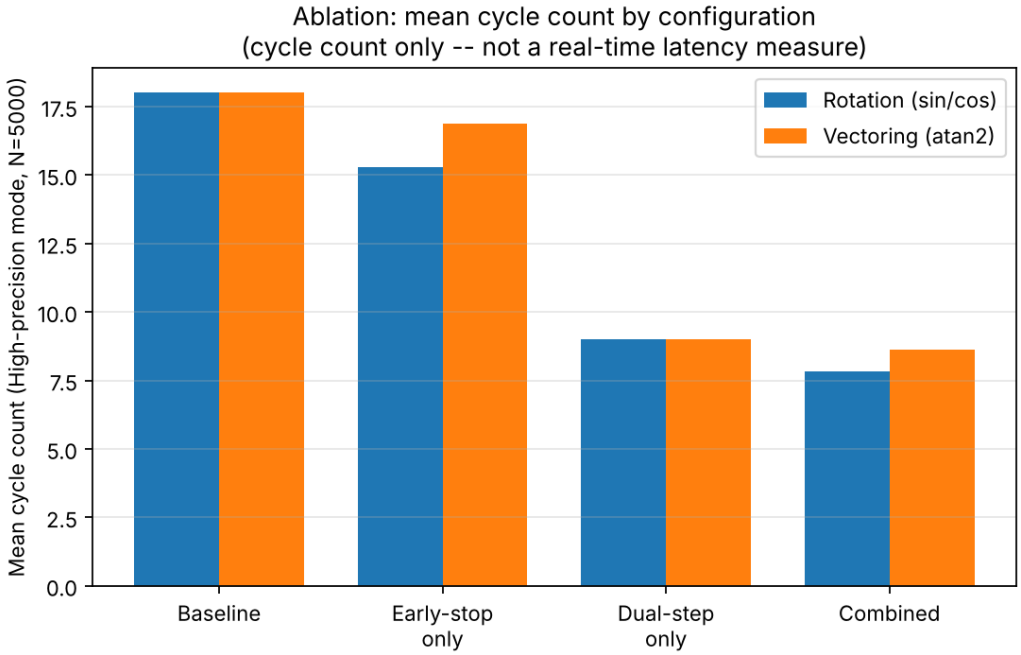


Figure 4: Ablation study (High-precision mode): mean cycle count for each configuration, both stages. Cycle count alone is *not* a latency measure – compare against the real (ns-based) throughput in Figure 6, Section 4.9, where the ranking reverses.

deployment with inputs of near-uniform magnitude well above the dynamic-range floor sees a tight guarantee, while one with inputs spanning down toward the floor sees the threshold degrade exactly as characterized in Section 2.2.

Beyond the maximum. Table 6 reports only the maximum observed error over the held-out set, which is a single order statistic and does not by itself characterize the error distribution. We therefore also computed the mean, root-mean-square (RMS), 99th-percentile, and 99.9th-percentile error over the same 5,000-vector held-out set. Because dual-stepping changes only cycle scheduling and not which iterations execute, Baseline and Dual-step-only have numerically identical error distributions in every mode (confirmed exactly, to the last represented bit, over all 5,000 held-out vectors); likewise Early-stop and Combined share identical error distributions with each other. Table 7 therefore reports only two distributions per mode – "No early-term." (Baseline/Dual-step-only) and "Early-term." (Early-stop/Combined) – rather than four.

Two patterns emerge that the maximum alone does not show. First, in rotation mode the direction is mode-dependent: at High precision, early termination raises the error at *every* statistic, mean included ($4.27\times10^{-6} \rightarrow 7.12\times10^{-6}$), consistent with Table 6's max increasing in the same mode ($1.49\times10^{-5} \rightarrow 2.16\times10^{-5}$); at Medium and Low precision, early termination instead *lowers* the mean/RMS/P99 (e.g. Low mode mean $4.35\times10^{-4} \rightarrow 2.91\times10^{-4}$), even in Medium mode where Table 6's max is unchanged to three significant figures – i.e. the bulk of the Medium-mode distribution shifts lower under early termination even though its worst sampled point does not. Second, in vectoring mode, High and Medium precision show early termination *lowering* the mean/RMS/P99 while simultaneously *raising* the P99.9/max (confirming the small-x_i relative-error effect discussed above is confined to the extreme tail – the rare small-magnitude inputs near the dynamic-range floor – not the bulk of the distribution), whereas Low mode shows early termination lowering every statistic including the mean. This distinction matters operationally: a system budgeting for *typical-case* error should use the mean/RMS columns here, while a system with a hard per-operation error bound must still budget for the P99.9/max columns in Table 6 regardless of what the mean suggests. **This cycle-count picture is, however, incomplete on its own.** Section 4.9 carries all four configurations through real technology mapping and static timing analysis, which shows that DSPC's shorter cycle count comes at the cost of a substantially longer critical path – long enough that the combined configuration's real (nanosecond) throughput ends up *lower*, not

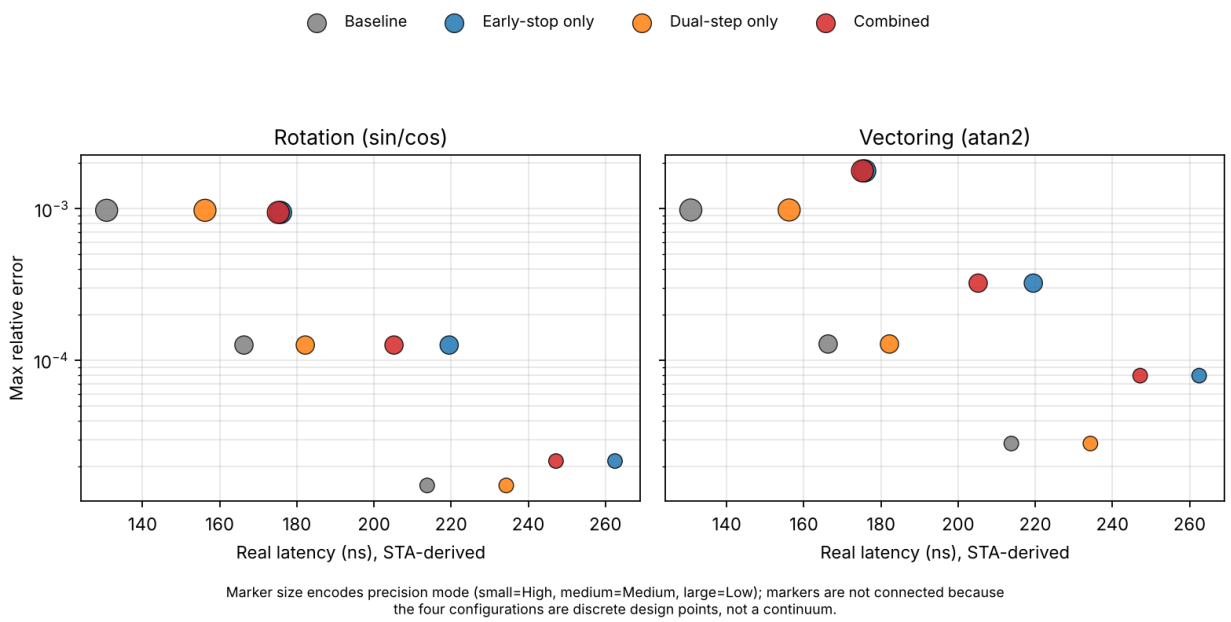


Figure 5: Accuracy-vs-real-latency operating points for all four configurations across the three precision modes, plotted against STA-derived latency (ns) rather than cycle count. Marker size encodes precision mode; points are deliberately left unconnected since the four configurations are discrete design points rather than a tunable continuum. Baseline Pareto-dominates all three modified configurations at every precision mode in both stages – for a given error level, Baseline is always the lowest-latency point – which is the real-time counterpart to the per-mode percentages reported in Section 4.9.

higher, than the fixed-depth baseline. We defer the full explanation to Section 4.9 rather than treat the cycle counts above as a latency claim.

Table 7: Full error distribution (not just the maximum), held-out 5,000-vector set, both stages. "No early-term." covers Baseline/Dual-step-only (identical distributions); "Early-term." covers Early-stop/Combined (identical distributions).

Stage	Mode	Config.	Mean	RMS	P99	P99.9
Rot.	High	No early-term.	4.27×10^{-6}	4.99×10^{-6}	1.15×10^{-5}	1.36×10^{-5}
		Early-term.	7.12×10^{-6}	8.26×10^{-6}	1.71×10^{-5}	2.01×10^{-5}
	Medium	No early-term.	5.57×10^{-5}	6.43×10^{-5}	1.18×10^{-4}	1.24×10^{-4}
		Early-term.	2.94×10^{-5}	3.49×10^{-5}	9.12×10^{-5}	1.21×10^{-4}
	Low	No early-term.	4.35×10^{-4}	5.08×10^{-4}	9.42×10^{-4}	9.73×10^{-4}
		Early-term.	2.91×10^{-4}	3.59×10^{-4}	8.07×10^{-4}	9.07×10^{-4}
Vec.	High	No early-term.	4.00×10^{-6}	4.74×10^{-6}	9.76×10^{-6}	1.31×10^{-5}
		Early-term.	2.98×10^{-6}	4.66×10^{-6}	1.52×10^{-5}	4.63×10^{-5}
	Medium	No early-term.	6.08×10^{-5}	7.04×10^{-5}	1.21×10^{-4}	1.24×10^{-4}
		Early-term.	4.69×10^{-5}	5.74×10^{-5}	1.11×10^{-4}	1.65×10^{-4}
	Low	No early-term.	4.87×10^{-4}	5.61×10^{-4}	9.68×10^{-4}	9.77×10^{-4}
		Early-term.	4.27×10^{-4}	5.07×10^{-4}	9.20×10^{-4}	9.70×10^{-4}

4.5. Synthesized Logic Cost

Each configuration was first synthesized with Yosys [18], an open-source Verilog synthesis suite, using a technology-independent generic-cell flow (`synth -top cordic_top`), then technology-mapped to the real SkyWater SKY130 high-density standard cell library (`sky130_fd_sc_hd`, typical corner, 25 °C, 1.8 V) via `dfflibmap`/`abc -liberty` – the same synthesis engine used in the open-source OpenLane RTL-to-GDSII flow [19, 20]. Table 8 reports the generic-cell counts (technology-independent, useful as a relative complexity measure) alongside the SKY130-mapped cell count and chip area for all four configurations. All configurations map to the same 120 flip-flops; the combinational cell count and area grow with DSPC, reflecting the added second micro-iteration datapath, both-candidate residual comparators, and 2:1 selection between one-step and two-step write-back.

Table 9 breaks down representative combinational cell types for the Baseline and Combined configurations after SKY130 mapping. The growth is concentrated in two-input Boolean gates (`nand2`, `nor2`, `xor2`, `xnor2`) and the 2:1 multiplexers (`mux2`) used for one-step/two-step write-back selection, consistent with DSPC duplicating the per-iteration adder/subtractor logic rather than adding new sequential state. The static leakage power in Table 8 scales with this cell growth (baseline 6.7 nW to combined 10.1 nW, a 50% increase, roughly tracking the 54% area increase); this is a state-averaged leakage figure derived directly from the liberty file's per-cell leakage entries and does not include switching (dynamic) power. Section 4.6 reports the activity-annotated dynamic-power analysis that completes this picture.

4.6. Dynamic (Switching) Power

The leakage figures above are state-averaged and do not reflect the switching activity of an operating core. To close this gap we built a gate-level, activity-annotated power analysis directly on the four SKY130-mapped netlists from Section 4.5, rather than assuming a generic toggle-rate. Each mapped netlist was simulated with Icarus Verilog against the SkyWater-released `sky130_fd_sc_hd` behavioral cell models, driven by the same representative rotation (sine/cosine) and vectoring (arctangent) stimulus set used for the accuracy evaluation in Section 4.3, cycling through all three precision modes. The resulting value-change dump (VCD) was loaded into OpenSTA with `read_vcd` and scored with `report_power`, which combines the annotated per-pin switching activity with the same liberty power tables used for the static analysis. Activity annotation was complete for all four configurations (9,438–14,801 pin activities per run, 0 unannotated), so no default-activity fallback was needed. Table 10 reports the resulting internal, switching, and leakage power.

Total power grows monotonically with the same cell-count/area trend as Table 8: Combined draws 46% more total power than Baseline (2.27 mW vs. 1.56 mW), driven mostly by switching power nearly doubling (0.53 to 0.96 mW) as the duplicated per-iteration adder/subtractor and comparator logic toggles on every operation. Dual-step-only accounts for most of this increase on its own (2.18 mW, only 4% below Combined), consistent with it contributing the larger share of the area growth in Table 8. Early-stop-only, by contrast, adds only 4% total power over Baseline (1.62 mW), in line with its much smaller area footprint. These figures are necessarily workload-dependent – they reflect the specific stimulus mix used here, not a worst-case or average-case bound over all possible input sequences – but they are real activity-driven measurements rather than an assumed toggle rate, and they confirm that DSPC's area and critical-path costs (Sections 4.5 and 4.9) carry a proportional dynamic-power cost as well.

Table 8: Post-synthesis cell count, area, and static leakage power. "Generic" is the technology-independent count before library mapping; "SKY130" is after mapping to `sky130_fd_sc_hd` (tt, 025C, 1.8V). Flip-flop count is 120 in all four configurations, both flows. Leakage power is summed from the actual per-instance liberty values in each mapped netlist (state-averaged per cell).

Configuration	Generic	SKY130 cells	Area (μm^2)	Leakage (nW)
Baseline	3,218	2,491	17,702.0	6.7
Early-stop only	3,542	2,579	18,686.7	7.0
Dual-step only	5,045	3,797	25,735.9	9.7
Combined	5,565	4,005	27,229.9	10.1

Table 9: Representative SKY130-mapped cell-type usage, Baseline vs. Combined.

Cell type	Baseline	Combined
`nand2_1`	309	551
`nor2_1`	296	555
`xor2_1`	157	217
`xnor2_1`	157	314
`mux2_1`	18	32
`and2_0`	29	49

4.7. FPGA-Targeted Resource Utilization

In addition to the SKY130 ASIC flow, we synthesized all four configurations with Yosys's `synth_ice40` flow, which maps to real Lattice iCE40 4-input LUTs, carry cells, and flip-flops rather than technology-independent generic gates. Table 11 reports the resulting resource counts. We report this as an FPGA-oriented logic-synthesis resource estimate, not a place-and-routed bitstream result on physical FPGA hardware; no FPGA-vendor Fmax or on-board measurement is claimed.

4.8. FPGA Place-and-Route: Pin Budget and Real Post-Route F_{max}

Table 11 is a logic-synthesis estimate only, as stated above; it does not place or route a bitstream and so cannot report a real F_{max}. We attempted to close this gap with `nextpnr-ice40`, targeting `iCE40HX8K-CT256` – the largest iCE40 device/package combination supported by the open-source Yosys/nextpnr/IceStorm toolchain. This surfaced a genuine interface-capacity limit rather than a timing result: `cordic_top`'s native parallel interface (32-bit `theta_in`, `x0_in`, `y0_in`, `cos_out`, `sin_out`, `angle_out`, plus `op`, `prec_sel`, `start`, `valid`, `cycle_count`) requires 209 single-bit I/O pins. The CT256 package, the largest in the IceStorm chip database, provides exactly 207 user I/O pins – two short. No iCE40 device/package pair in the open-source toolchain can bond this interface directly, so a placed-and-routed bitstream of the native top-level module cannot be built on this device family.

To still obtain a real, placed-and-routed F_{max} for the engine logic itself, we wrapped the *unmodified* `cordic_top` / `cordic_engine` core in a minimal test harness (`cordic_fpga_shell.v`) that loads the three input operands and reads back the three output operands through an 8-bit shift register interface, reducing the pin count to 39. This is a standard pin-reduction technique for bringing up wide-datapath IP on pin-limited FPGA packages; no CORDIC arithmetic is modified, only plain shift-register load/read logic is added around the core. We confirmed from nextpnr's critical-path report that the posedge-to-posedge critical path in every configuration lies entirely inside `u_engine` (the CORDIC core), not in the shift-register shell, so the reported F_{max} reflects the engine's own routed timing rather than an artifact of the harness. Table 12 reports the resulting placed-and-routed logic-cell utilization and F_{max} for all four configurations.

The FPGA result reproduces, on an entirely different fabric and toolchain, the same qualitative finding as the SKY130/OpenSTA result in Table 13: early-stop and dual-step each substantially reduce F_{max} relative to baseline (60.3 MHz to 33.0 MHz and 32.1 MHz, respectively), and the combined configuration is worst of all (22.5 MHz, 2.7× slower than baseline). The logic-cell count also grows monotonically from baseline to combined (1,832 to 3,152, a 72% increase), consistent with the added termination-check and dual-candidate-evaluation logic reflected in

Table 10: Gate-level, VCD-activity-annotated power, SKY130 tt/025C/1.8V corner, from representative rotation/vectoring stimuli across all three precision modes. Leakage matches Table 8's state-averaged figures to within the small difference expected between a fixed reference state and activity-weighted averaging.

Configuration	Internal (mW)	Switching (mW)	Leakage (nW)	Total (mW)
Baseline	1.03	0.53	6.6	1.56
Early-stop only	1.06	0.56	7.1	1.62
Dual-step only	1.29	0.90	9.8	2.18
Combined	1.32	0.96	10.2	2.27

Table 11: FPGA-targeted logic synthesis resource utilization (Yosys `synth_ice40`, Lattice iCE40 4-input LUT architecture).

Configuration	LUT4	Carry	Flip-flops
Baseline	1,573	628	120
Early-stop only	1,760	694	120
Dual-step only	2,537	887	120
Combined	2,779	1,019	120

the ablation cycle counts of Table 6. We do not compute an FPGA-side operations-per-second figure analogous to Table 15 because the shift-register load/read sequencing in the test harness is not representative of a real operand-feed datapath and would conflate harness I/O latency with engine latency; the F_{max} comparison above is the portion of this result we consider trustworthy for the harness used.

Table 12: Real placed-and-routed iCE40HX8K-CT256 results (`nextpnr-ice40`, via the pin-reduction test harness of Section 4.8; the harness itself is excluded from the reported critical path, verified via nextpnr's critical-path report).

Configuration	ICESTORM_LC	LC util. (%)	F_{max} (MHz)
Baseline	1,832	23%	60.3
Early-stop only	2,018	26%	33.0
Dual-step only	2,868	37%	32.1
Combined	3,152	41%	22.5

4.9. Real Timing-Closed Throughput (OpenSTA)

The results so far – cycle counts (Table 6) and technology-mapped area (Table 8) – do not by themselves establish a real-time latency claim. To close this gap, we built OpenSTA [24] from source against the same `sky130_fd_sc_hd` liberty file (the ABC-internal `stime` pass used in an earlier version of this study failed to parse several clock-related cells in the mapped netlist; OpenSTA's full liberty reader does not have this limitation) and ran static timing analysis directly on each SKY130-mapped netlist from Section 4.5, with a stress-test clock period short enough to report the true critical path via worst negative slack. Table 13 reports the resulting critical path delay and maximum clock frequency for all four configurations.

The critical path grows substantially with DSPC – more than doubling from baseline to dual-step-only (11.88 ns to 26.03 ns) – because the second micro-iteration is chained combinationally after the first, and the termination check must additionally evaluate the residual/floor/target condition against *both* candidate states before selecting the write-back mux. Combining the per-mode cycle counts from Table 6 with the F_{max} values above gives the real operation latency and throughput in Table 15.

4.9.1. PVT corner sweep

Table 13 reports only the typical (tt) corner. To check that the relative ranking among configurations is not an artifact of a single operating point, we re-ran the same OpenSTA methodology against the fast (ff, −40 °C, 1.95 V) and slow (ss, 100 °C, 1.6 V) SKY130 sign-off corners, using the real per-corner liberty timing tables released by SkyWater for `sky130_fd_sc_hd` (re-serialized from the PDK's own JSON release format into the same plain-text liberty the tt corner uses; no timing or power values are altered or interpolated). Table 14 reports the resulting critical path and F_{max} at all three corners for all four configurations.

The ff-to-ss spread (3.3–3.75× critical path depending on configuration) is in line with the typical multi-× speed range reported between SKY130's fast and slow sign-off corners, and – more importantly for this paper's argument – the *relative ranking among the four configurations is unchanged across all three corners*: Baseline is fastest and Combined is slowest at ff and ss exactly as at tt, and the pairwise ratios are close to corner-invariant (e.g. Combined/Baseline critical path is 2.53× at tt, 2.33× at ff, and 2.63× at ss). The throughput reversal reported above for the tt corner is therefore not a tt-specific artifact; it would be observed at the fast and slow sign-off corners as well.

The result is the opposite of what the cycle-count table alone suggests, and the ranking among the three modified configurations is itself counter-intuitive. Despite executing 54%, 51%, and 47% fewer cycles, the Combined configuration's real throughput is *13.4%, 19.0%, and 25.4% lower* than the Baseline's for High, Medium, and Low precision, respectively, because F_{max} drops by 60.5% (84.2 to 33.3 MHz) – proportionally more than the cycle count drops. Of the two mechanisms in isolation, *dual-step-only comes closest to preserving Baseline's real throughput* (8.8%, 8.7%, and 16.3% lower for High/Medium/Low, despite a 45–50% cycle-count reduction and *zero* added numerical error), because although its F_{max} also drops substantially (84.2 to 38.4 MHz), the halved cycle count very nearly compensates. By contrast, *early-stop-only is the worst performer of the three* – 18.6%, 24.1%, and 25.6% lower throughput than Baseline – even though it leaves the per-iteration combinational depth essentially unchanged and only modestly reduces cycle count (2–11%, Section 3): its F_{max} nonetheless drops from 84.2 to 61.3 MHz (a 27% reduction) because the added residual/floor/target comparator logic sits on the critical path, and this frequency loss outweighs its small cycle savings by a wide margin. *Neither mechanism, alone or combined, improves on the fixed-depth baseline's real throughput at any precision mode in this implementation.* The practical benefit this design does deliver is the ability to select a shallower fixed iteration depth at runtime (18/14/11 cycles for High/Medium/Low, Table 6, Baseline rows) – i.e., choosing a lower precision mode outright – which both reduces cycles and leaves F_{max}

Table 13: OpenSTA-derived critical path delay and maximum frequency, SKY130 tt/025C/1.8V corner.

Configuration	Critical path (ns)	F_{max} (MHz)
Baseline	11.88	84.2
Early-stop only	16.32	61.3
Dual-step only	26.03	38.4
Combined	30.05	33.3

Table 14: OpenSTA-derived critical path and F_{max} across the fast (ff, −40 °C, 1.95 V), typical (tt, 025 °C, 1.8 V), and slow (ss, 100 °C, 1.6 V) SKY130 `sky130_fd_sc_hd` corners.

	ff (−40°C, 1.95V)		tt (025°C, 1.8V)		ss (100°C, 1.6V)	
Configuration	CP (ns)	F_{max}	CP (ns)	F_{max}	CP (ns)	F_{max}
Baseline	7.22	138.5	11.88	84.2	24.03	41.6
Early-stop only	9.46	105.7	16.32	61.3	33.73	29.6
Dual-step only	14.98	66.8	26.03	38.4	53.60	18.7
Combined	16.83	59.4	30.05	33.3	63.16	15.8

unchanged, unlike either the early-termination or dual-step mechanism. We discuss why DSPC and early termination behave this way under real timing closure, and what would need to change for either to pay off, in Section 5.1.

4.10. Word-Length Sensitivity

Because the target depths and thresholds in Table 3 were tuned at a single fixed-point format (Q4.20, 24-bit nominal operands, 32-bit internal registers), we additionally swept the fractional word length (FRAC = 14–24 bits) against several fixed iteration depths to characterize where accuracy becomes word-length-limited rather than iteration-limited. Figure 7 shows the result for the rotation stage (500 randomized angles).

At 20-bit fractional precision (the format used in Sections 3–4), the two deeper depths shown behave differently as FRAC increases further: at depth 18, error falls by less than 2× from FRAC=20 to FRAC=24, consistent with the design having nearly reached the point where *iteration count*, not *register width*, is the binding constraint; at depth 20, by contrast, error continues to fall by roughly 4× over the same FRAC range, indicating that at this deeper depth word-length still contributes measurably to the error budget and the design has *not* yet reached that point. We do not extend the "iteration-count-limited" conclusion to depth 20 or to the High-precision mode generally; it applies specifically to depth 18 and shallower. At lower depths (12), increasing FRAC beyond 16–18 bits gives essentially no improvement, since the 2^{-12}-scale truncation of the *algorithmic* (iteration-count-limited) error already dominates the smaller $2^{-\mathrm{FRAC}}$-scale rounding error. This confirms that the Low-precision mode's accuracy floor in Table 6 is set primarily by its target depth choice rather than by the 24-bit fixed-point format, and that reducing the *fractional* word length to as low as FRAC=16 (within the swept range of Figure 7, and well short of the full 24-bit operand) would not measurably degrade Low-mode accuracy – a possible avenue for additional area reduction at low precision that this design does not currently exploit. The depth=20 curve in Figure 7 is not used by any of the three operating precision modes (Table 3); we include it to show that, one iteration beyond the High-precision mode's actual depth of 18, the design has *not* yet reached the iteration-count-limited regime, which is consistent with depth 18 itself sitting close to, but not exactly at, that boundary.

Table 15: Real (STA-derived) latency and throughput per precision mode. Cycle counts are the rotation/vectoring average from Table 6.

Config.	Mode	Cyc.	Lat. (ns)	Tput. (Mops/s)
Baseline	High	18.00	213.8	4.68
	Medium	14.00	166.3	6.01
	Low	11.00	130.7	7.65
Early-stop only	High	16.09	262.5	3.81
	Medium	13.45	219.5	4.56
	Low	10.77	175.8	5.69
Dual-step only	High	9.00	234.3	4.27
	Medium	7.00	182.2	5.49
	Low	6.00	156.2	6.40
Combined	**High**	8.23	247.2	**4.05**
	Medium	6.83	205.2	**4.87**
	Low	5.83	175.2	**5.71**

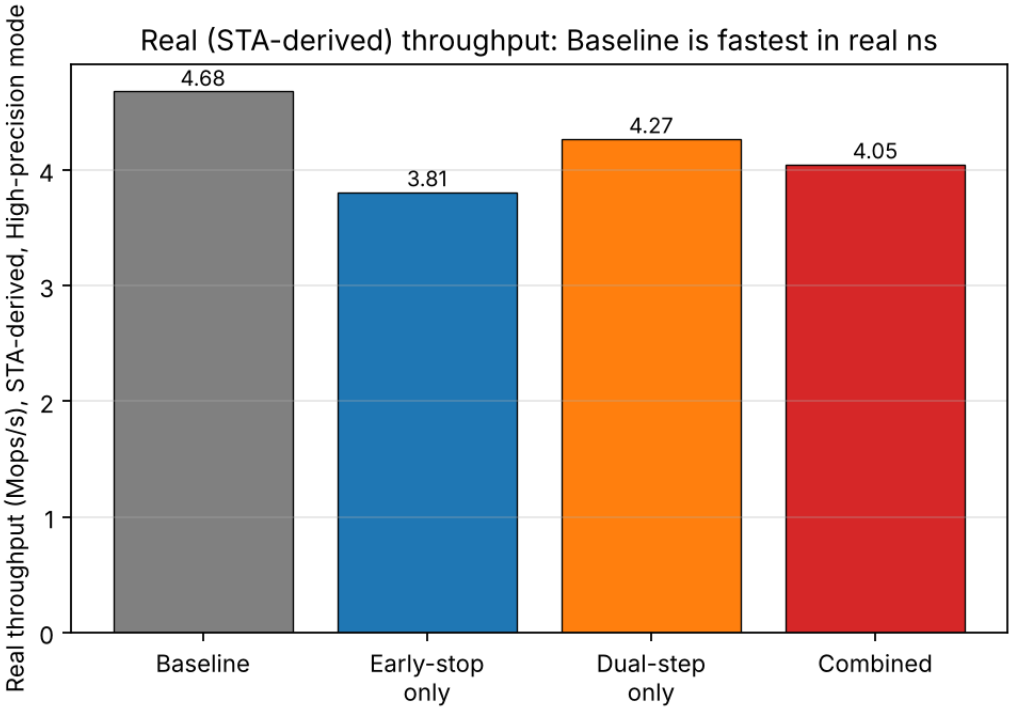


Figure 6: Real (STA-derived) throughput in Mops/s, High-precision mode, for all four configurations – the real-nanosecond counterpart to the cycle-count comparison in Figure 4. Unlike the cycle-count picture, Baseline is fastest in real time; among the modified configurations, Dual-step-only is the closest runner-up, and Early-stop-only is the slowest.

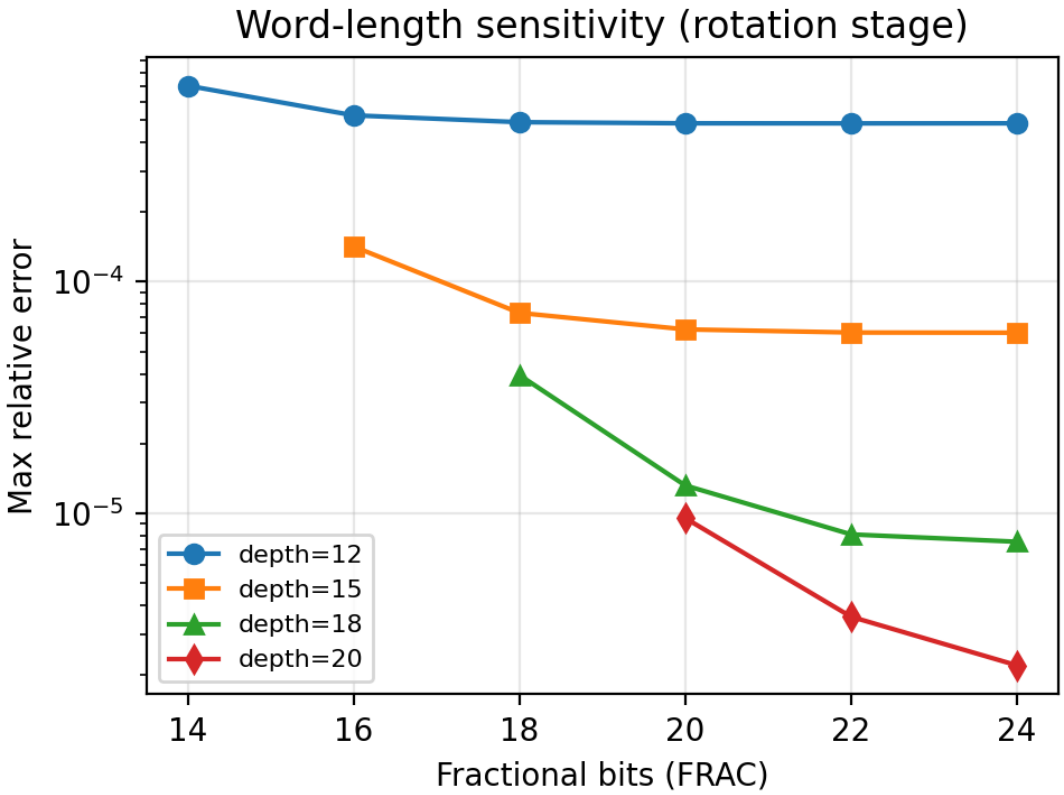


Figure 7: Word-length sensitivity: maximum relative error vs. fractional bit width, for four fixed iteration depths. Each depth's curve begins at the smallest swept FRAC value satisfying FRAC $\geq$ depth, since a fractional width narrower than the iteration depth cannot represent the final micro-rotation's shift amount; this is why, e.g., the depth=20 curve has no point left of FRAC=20 while depth=12 starts at FRAC=14 (the smallest value swept).

5. Discussion

5.1. What It Would Take for DSPC to Pay Off

5.1.1. Pipelined DSPC: a feasibility study

Section 4.9's finding – that DSPC reduces cycles but not real throughput in this implementation – is a statement about *this non-pipelined, combinationally-chained realization of the idea*, not necessarily a general refutation of dual-stepping. We tested the natural alternative directly rather than leaving it as speculation: we built a second variant of `cordic_engine` (`cordic_engine_pipe.v`, released alongside the main RTL) in which a register is placed between the two micro-iterations instead of chaining them combinationally, and synthesized and timed it with the same Yosys/SKY130/OpenSTA flow used throughout this paper. The result clarifies exactly what pipelining can and cannot buy here. Because iteration i+1 has a genuine data dependency on iteration i's committed state (Eqs. 1–3), a register placed between the two micro-iterations of a *single, non-overlapped* operation cannot preserve any part of DSPC's cycle advantage: the registered variant commits one iteration per clock, exactly like Baseline, and our RTL simulation of it confirms an identical cycle count (18/14/11 across High/Medium/Low, matching Baseline's Table 6 rows exactly). Synthesizing and timing this variant confirms the other half of the trade: its critical path (16.19 ns, 61.8 MHz) matches Early-stop-only's measured critical path (16.32 ns, 61.3 MHz, Table 13) to within 0.8% – consistent with its post-synthesis cell count and area (2,579 cells, 18,686.7 μm^2) being numerically identical to Early-stop-only's own synthesis result, i.e. the synthesis tool collapses the two descriptions to the same netlist. In short: for a single, latency-critical operation, pipelining DSPC's two micro-iterations recovers Baseline-class F_{max} by giving back *all* of DSPC's cycle-count advantage, not part of it – there is no intermediate point on this axis, because the data dependency, not the combinational chaining per se, is what forces the choice. A real cycle-count-*and*-throughput benefit from pipelining would instead require overlapping the iterations of *multiple, independent* CORDIC operations in flight at once (a classical pipelined/ streaming datapath with throughput of one iteration-slot per clock across many operations), which is a materially different system interface (multiple in-flight operations, not the single start/valid handshake used here) and a correspondingly larger redesign – including draining the pipeline correctly when early termination fires mid-flight for one of several in-flight operations – that we have not built or verified and leave as future work.

5.1.2. Deployment context: when cycle count alone is what matters

A second, independent path does not require pipelining at all. In a system where the clock period is already fixed by a slower, unrelated critical path elsewhere on the same die (a common situation for a CORDIC engine embedded in a larger SoC), *cycle* count – not this engine's own standalone F_{max} – is what determines its contribution to end-to-end latency, and DSPC's cycle reduction would be realized without penalty. This second condition, however, only holds if the system clock period is already at least as long as the Combined configuration's own critical path (30.05 ns, Table 13, i.e. an SoC-level clock at or below roughly 33 MHz); at any higher SoC clock frequency, the CORDIC engine's own critical path would itself become the binding constraint, and the real-throughput penalty characterized here would reappear. Neither condition holds in the standalone, maximum-frequency evaluation used throughout this paper, which is why we report the throughput reversal as the headline timing result rather than the cycle-count reduction.

5.2. DSPC vs. Higher-Radix CORDIC

Radix-4 and other higher-radix CORDIC variants also reduce cycle count by processing more angle per iteration, but require redundant digit selection and, in most reported designs, extra correction logic to bound the angle-recoding error [6]. DSPC instead keeps the underlying radix-2 recurrence unchanged and simply commits two micro-iterations per clock when the residual has not yet converged, which is why dual-step scheduling in this design costs zero additional numerical error (Table 6, identical error columns for Baseline and Dual-step only) – though, as Section 4.9 shows, that numerical-cost-free property does not by itself imply a real latency win, since the mechanism still roughly doubles the combinational logic (and hence critical path) of the single-step datapath.

Table 16: Qualitative comparison with representative circular-mode CORDIC architectures. "Adaptive" means the design exposes more than one runtime-selectable precision/latency operating point; "Mult.-free" means the datapath is shift-add only, with no embedded multiplier.

Work	Technique	Radix	Mult.-free	Adaptive
This work	Early-term. + DSPC	2	Yes	Yes (3)
Zhang *et al.*† [27]	Early-term. + PACK2	2	Yes	Yes (3)
Lin & Wu [12]	Mixed-scal.-rotation	2	Yes	No
Strollo [16]	Hybrid CORDIC-mult.	2	No	No
Aggarwal *et al.* [15]	Redesigned scale-free	2	Yes	No
Changela *et al.* [17]	Mixed-radix scale-free	4	Yes	No
Salehi *et al.* [8]	Angle-recoded LL	2	Yes	No
Xilinx LogiCORE [25]	Word-serial/par.	2	Yes	No*

*Configurable at synthesis/generation time (word length, architecture, iteration count), not selectable at runtime by a live hardware input the way `prec_sel` is here. †Unreviewed SSRN preprint, hyperbolic-mode Nth-root function (not circular-mode trigonometric), Intel HardCopy III FPGA target; included for architectural positioning only, not a matched-configuration PPA comparison (see discussion below).

5.3. Comparison Basis with Lookup-Table and Polynomial Methods

Lookup-table sine/cosine generators, as used in classical direct digital frequency synthesizers [23, 22], trade table depth directly for accuracy and are attractive at low precision but scale poorly in memory as accuracy requirements increase [8]; polynomial and piecewise-polynomial approximants (e.g. Horner-scheme minimax polynomials evaluated with multiply-accumulate hardware [21], hybrid piecewise-polynomial phase-to-amplitude converters [22], or hybrid CORDIC-multiplier synthesizer/mixer designs [16]) avoid the memory cost but reintroduce multiplier hardware, whose area and delay scale with the target word length in a way that shift-add CORDIC avoids. The proposed design retains the multiplier-free shift-add property of CORDIC across all three precision modes, trading only combinational logic (Table 8) for iteration count, which is a more graceful and runtime-reconfigurable trade-off than re-synthesizing a different lookup-table depth, polynomial order, or multiplier width for each target accuracy – though, per Section 4.9, the precision-mode selection itself (which does not enable DSPC's combinational doubling) is the part of this trade-off that actually delivers a real-time benefit; the DSPC scheduling mechanism should be read as a cycle-count/area trade-off whose real-time payoff depends on the deployment context discussed in Section 5.1, not as a standing latency improvement.

5.4. Qualitative Comparison with Representative CORDIC Architectures

A direct quantitative PPA comparison against prior circular-mode CORDIC designs would require matching word length, precision target, target technology, and speed grade to each cited work's own evaluation setup – conditions we cannot establish for designs we have not independently re-implemented on the SKY130/iCE40 flows used here. We therefore do not report side-by-side Fmax/area numbers pulled from other papers' own (differently configured) evaluations, since a table populated that way would create a false impression of a controlled comparison. Table 16 instead compares architectural *characteristics* that can be read directly off each work's own description, positioning this design within the design space rather than ranking it by unmatched numbers.

By this comparison, the adaptive-precision mechanism (Section 3) is what differentiates this design architecturally – runtime precision/latency selection via a single input, rather than a synthesis-time choice – while DSPC (Section 4.2), per Section 4.9's finding, is not shown here to be a net throughput advantage over the simpler single-step datapath that most of these prior designs already use; nor, contrary to our own initial expectation, is residual-based early termination alone (Section 3), which is in fact the *larger* of the two timing penalties observed. Zhang *et al.* [27] propose essentially the same early-termination-plus-dual-step framework – the same High/Medium/Low depth/floor/threshold structure, a termination check evaluated on the candidate next state, and two micro-iterations merged into one clock cycle (their PACK2, our DSPC) – for a different function (Nth-root via hyperbolic vectoring, linear vectoring, and hyperbolic rotation, rather than circular-mode sine/cosine/arctangent) and a different target technology (Intel HardCopy III FPGA at a reported single F_{max} of 146.69 MHz, versus the SKY130 ASIC flow used here). We do not compare their reported numbers against ours directly in Table 16 because the functions, word lengths, and targets differ; however, one aspect of their reported methodology is worth noting explicitly. Their throughput figures are obtained by dividing the same 146.69 MHz across every configuration by that configuration's mean cycle count (e.g. 146.69/84.0 cycles for their baseline and 146.69/51.0 cycles for PACK2-only), which implicitly assumes

Table 17: Quantitative comparison with AMD/Xilinx LogiCORE CORDIC IP [25] (32-bit, word-serial, rotate mode, data sheet [26], Table 30/33), alongside this design's real placed-and-routed iCE40 result (Baseline, Table 12). See caveats below before drawing conclusions from this table.

Design	Device/process	LUT arch.	Logic	FF	F_{max} (MHz)
Xilinx CORDIC IP	Virtex-7 (28 nm)	6-input LUT	1,104 LUT	837	232
Xilinx CORDIC IP	Kintex-7 (28 nm)	6-input LUT	1,100 LUT	837	230
This work	iCE40HX (40 nm)	4-input LUT	1,832 LC*	(incl. above)	60.3

*iCE40 `ICESTORM_LC` is a combined LUT4+flip-flop logic cell (nextpnr/IceStorm's reporting unit), not a separate LUT count, so the 1,832 figure is not directly additive with the Xilinx LUT/FF columns.

that neither PACK2 nor early termination changes F_{max} at all. Our own post-synthesis timing analysis (Section 4.9) shows this assumption does not hold in our flow: the equivalent mechanisms measurably reduce F_{max} here, enough to erase the cycle-count benefit entirely. We report this respectfully as a methodological caution rather than a critique of [27]'s specific numbers, which we have not independently reproduced on their target technology – but it suggests that a cycle-count-only throughput claim for this class of scheduling mechanism should be treated carefully until confirmed by real timing closure on the target technology, whatever that technology is. Vendor CORDIC IP such as AMD/Xilinx's LogiCORE CORDIC [25], configurable for rotation, vectoring, and several other functions with either word-serial or fully parallel architectures, remains a natural practical reference point: its own architecture description (a coarse quadrant rotation into the first quadrant, matching the folding stage in Section 2.2) is methodologically close enough to this design that a direct resource/frequency comparison is worth attempting, which we do quantitatively below using the vendor's own published datasheet numbers.

5.5. Quantitative Comparison with Vendor CORDIC IP

A fully matched PPA comparison against the five academic prior works in Table 16 is not possible from their published text alone: Lin & Wu [12], Strollo [16], Aggarwal *et al.* [15], Changela *et al.* [17], and Salehi *et al.* [8] are each behind a publisher paywall with no open-access copy, and automated retrieval of their results tables was not possible; we therefore do not report numbers for them rather than approximate figures we could not independently verify, consistent with this paper's standing policy of reporting only measured or directly-sourced values. AMD/Xilinx's LogiCORE CORDIC IP [25] is different in this respect: its performance-characteristic tables are published directly by the vendor (data sheet [26], the companion resource/timing reference for the PG105 product guide) and are reproduced here verbatim for the configuration closest to this design's own word length and architecture – 32-bit, word-serial, rotate mode, auto-iteration/auto-precision, coarse rotation enabled, maximum pipelining. Table 17 places these numbers alongside this design's own real, placed-and-routed iCE40 result (Table 12, Baseline configuration, which is architecturally the closest match: a single micro-rotation per cycle, no DSPC, no early termination).

Three caveats apply to Table 17, and we state them plainly because they limit how much can be concluded from it. *LUT granularity.* Xilinx 7-series logic cells use 6-input LUTs with dedicated carry-chain hardware; iCE40 uses 4-input LUTs with a simpler carry primitive, so a single Xilinx LUT typically packs more logic than a single iCE40 LUT4/`ICESTORM_LC` – the logic-cell counts are not on a common footing, and we report them side by side rather than computing a ratio. *Process and toolchain.* Xilinx's numbers come from a mature, closed, vendor-optimized place-and-route flow on a 28 nm commercial process; ours come from the open-source Yosys/nextpnr/IceStorm flow (Section 4.8) on an older, smaller 40 nm iCE40 process with no vendor-grade timing optimization – both the 3–4× F_{max} gap and any area difference plausibly reflect this toolchain/process gap at least as much as an algorithmic one, and we make no claim to have isolated the two. *Configuration match.* The compared Xilinx case is auto-iteration/auto-precision with coarse rotation, which is architecturally close to but not identical to this design's fixed Q4.20, three-mode datapath; we selected it as the closest published word-length and architecture match available in the datasheet, not as a certified apples-to-apples configuration. With these caveats stated, the table supports one specific, modest claim: this design's real, placed-and-routed F_{max} is within the same order of magnitude as a mature commercial CORDIC IP core at a comparable word length and architecture, on a smaller and older open-source-toolchain FPGA target, rather than orders of magnitude slower as a naive RTL simulation-only comparison might suggest.

5.6. Application-Level Validation: DDS/NCO Spectral Purity

The per-sample error statistics in Table 6 characterize accuracy at individual operating points but do not directly answer the question an application designer actually asks: if this engine is used as a sine/cosine generator – its canon-

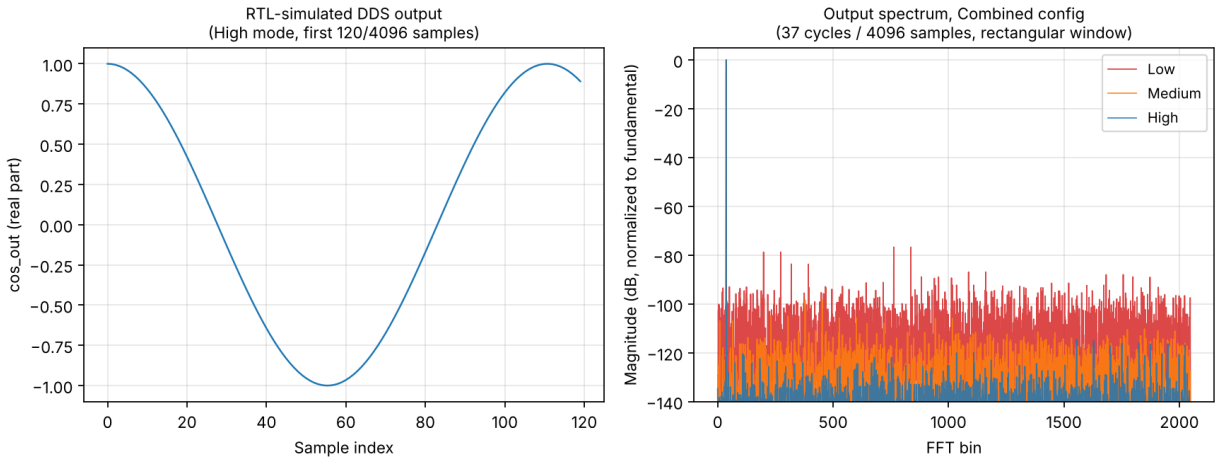


Figure 8: Application-level validation: RTL-simulated DDS/NCO output waveform (left) and output spectrum at all three precision modes (right), Combined configuration, 37-cycle coherent phase ramp over 4,096 samples.

Table 18: DDS/NCO spectral purity, Combined configuration, from real RTL simulation (37 cycles / 4,096 samples, coherent sampling, rectangular window).

Precision mode	SFDR (dBc)	THD (dB)
High	114.7	−121.8
Medium	98.3	−110.3
Low	76.7	−106.7

ical application, as a direct digital synthesizer (DDS) or numerically-controlled oscillator (NCO) core – how clean is the resulting waveform in the frequency domain? We answer this with a coherent-sampling spectral test, the standard methodology for characterizing DDS and data-converter spectral purity. We drove the real RTL (`cordic_top`, Combined configuration, rotation mode) with a phase ramp of 4,096 samples spanning exactly 37 cycles (37 and 4096 are coprime, so the tone falls exactly on FFT bin 37 with no spectral leakage, avoiding the need for a tapering window), via RTL simulation (`iverilog`) identical in kind to the methodology already used throughout this paper – the same simulator, the same `cordic_top` instantiation, only a different input sequence and a spectral rather than per-sample analysis of the output. We repeated this at each of the three precision modes and computed, from the real simulated `cos_out` sequence, the spurious-free dynamic range (SFDR, the ratio of fundamental power to the largest non-fundamental spur) and total harmonic distortion (THD, summed power in the second through eighth harmonic bins relative to the fundamental). Figure 8 shows the time-domain output and the resulting spectra; Table 18 reports the SFDR/THD figures.

SFDR and THD both degrade monotonically from High to Low precision mode, tracking the per-sample error floors in Table 6 (High's roughly 10^{-5} worst-case error against Low's roughly 10^{-3}). Using the standard single-tone relationship ENOB $\approx$ (SFDR -1.76)/6.02, the three modes correspond to approximately 18.8, 16.0, and 12.4 effective bits respectively – i.e. `prec_sel` functions, from an application standpoint, as a runtime-selectable trade between generated waveform quality and cycle count/energy, exactly the trade the adaptive-precision mechanism (Section 3) is intended to expose. This is a single representative application (sinusoid generation via the rotation path); it does not substitute for a full system-level validation in a specific end application (e.g. an actual phase-locked loop or motor resolver-to-digital converter), which remains future work.

Robustness to frequency and sampling conditions. The 37-cycle test above is one operating point (one output frequency, one sampling condition). We re-ran the identical RTL simulation and analysis at two more points to check how much of the result is specific to that choice. First, a much higher coherent output frequency (811 cycles over the same 4,096 samples, still coprime with 4,096 so no leakage): SFDR is *identical* to three significant figures at every

precision mode (114.7/98.3/76.7 dBc for High/Medium/Low, matching Table 18 exactly), confirming that the measured noise floor is set by the engine's own fixed-point quantization and iteration-count error – which do not depend on output frequency – rather than by any frequency-dependent artifact specific to the 37-cycle test. Second, a *non-coherent* case (37.3 cycles over 4,096 samples, so the tone does not fall on an exact FFT bin and a Hanning window is required to control the resulting leakage, following standard spectral-test practice): SFDR collapses to approximately 35.9 dBc at *every* precision mode (35.86, 35.86, and 35.85 dBc for High, Medium, and Low respectively), erasing the 38 dB spread between modes that the coherent test cleanly shows. This is not a property of the engine – it is the Hanning window's own sidelobe leakage (on the order of −32 dB relative to the main tone) dominating the measurement and masking the much smaller differences between precision modes entirely. We report this because it is a genuine methodological finding in its own right: a non-coherent spectral test of this engine, without a higher-dynamic-range window (e.g. Blackman-Harris) or a longer/coherent-redesigned sample count, would substantially *understate* how much the precision modes actually differ, which is why Table 18 uses the coherent-sampling methodology throughout rather than an arbitrary sampling ratio.

5.7. Limitations

Several simplifying choices and scope boundaries should be stated plainly. *Scope.* This paper addresses circular-mode CORDIC (sine, cosine, arctangent) only; the adaptive-precision and DSPC mechanisms are not evaluated here for hyperbolic-mode functions, where the repeated-index correction required for convergence [5] would interact with DSPC's paired-step scheduling in ways this study does not characterize. *Input range.* The quadrant folding in Section 2.2 assumes the input angle is already wrapped to $[-\pi, \pi)$; a full mod-2π reduction stage for unbounded angle inputs is standard but not included in the area/cycle figures reported here. *Usable operand magnitude.* Section 4.3's boundary testing found that vectoring inputs with magnitude below roughly 2^{-13} in this Q4.20 format produce rapidly growing absolute error (Section 4.3); consequently, applications that vectorize operands routinely near this floor – e.g. small-signal sensor residuals or post-subtraction values in a control loop – should apply input normalization (rescaling to a larger representable magnitude before the CORDIC stage) or a wider internal representation; applications whose inputs stay well above this floor (most angle-tracking and phase-generation uses) are unaffected. *Offline-tuned thresholds.* The three precision modes are runtime-selectable at the hardware interface (via `prec_sel`), but the depth/floor/threshold values themselves (Table 3) are fixed constants chosen by an offline sweep against representative test data (Section 3.2), not adapted online to the actual input distribution or observed error at run time; this choice is not seed-fragile (Table 4 shows the same triple would be selected from five independent tuning draws), but it is still fixed once at design time rather than adaptive to a deployment's actual input statistics. *Input distribution.* The ablation study (Section 4.4) and the corner-case testing (Section 4.3) together cover uniform-random and deterministic boundary/axis inputs, but not a specific downstream application's input distribution (e.g. the near-zero-crossing-heavy angle distribution typical of a phase accumulator in a direct digital synthesizer); early-termination savings in particular are sensitive to the input distribution and may differ from Table 6 for a specific application. *Power.* Table 10 (Section 4.6) reports gate-level, VCD-activity-annotated dynamic power alongside static leakage, but the figures reflect the specific rotation/vectoring stimulus mix used there, not a worst-case or average-case bound over all possible input sequences; a full energy-per-operation figure for a specific deployment workload would require re-running the same VCD methodology against that workload's actual activity profile. *Process-voltage-temperature coverage.* Section 4.9 reports critical path and F_{max} at the typical (tt, 025 °C, 1.8 V), fast (ff, −40 °C, 1.95 V), and slow (ss, 100 °C, 1.6 V) SKY130 corners, but all three remain a single technology node; an advanced-node implementation was not attempted. *Register width.* The 120-flip-flop control/state overhead is dominated by the three 32-bit (x, y, z) registers required to avoid overflow (Section 4.3); the word-length sweep in Section 4.10 suggests a narrower fractional width would not measurably degrade Low-mode accuracy, but this design does not currently exploit that headroom.

6. Conclusion

This paper presented an adaptive-precision, dual-step CORDIC engine unifying circular-mode sine, cosine, and arctangent computation in a single reconfigurable datapath, transferring and critically re-evaluating – for this circular-mode setting and under real post-synthesis timing closure – a residual-based early-termination and dual-step scheduling framework previously proposed for hyperbolic-mode Nth-root CORDIC in an unreviewed preprint [27]. We evaluated the resulting design more rigorously than cycle-count simulation alone: a tuning/evaluation data split, a 10×

larger held-out random test set, a deterministic boundary/worst-case test set, real SKY130 and iCE40 synthesis, real per-cell leakage and gate-level activity-annotated dynamic power, and real OpenSTA static timing closure repeated across fast, typical, and slow SKY130 PVT corners. The boundary testing paid for itself immediately, surfacing a genuine branch-cut bug (a 2π output-angle error at exact-axis vectoring inputs, now fixed) and a precisely quantified fixed-point dynamic-range floor (operand magnitudes below 2^{-13} in the Q4.20 format). On the held-out evaluation set, combining residual-based early termination with dual-step-per-cycle (DSPC) scheduling reduces mean iteration count by 47–54% across three precision modes with error remaining within 1.8×10^{-3} even in the lowest-precision mode. **The central, and more important, finding of this study is that this cycle-count reduction does not survive contact with real post-synthesis timing, and that the two mechanisms are not equally bad:** OpenSTA analysis on the SKY130-mapped netlists shows both mechanisms lengthen the critical path enough that real throughput ends up 13–25% *lower* than a naive fixed-depth, single-step baseline for the Combined configuration, despite executing far fewer cycles. Counter-intuitively, it is *dual-step-only*, not early-termination-only, that comes closest to preserving the baseline's real throughput (9–16% lower); early-termination alone – despite adding *zero* numerical error and barely touching the per-iteration combinational depth – is in fact the *worst* performer of the three modified configurations (19–26% lower), because its residual-comparator logic still costs enough critical-path delay to erase its modest cycle savings several times over. The practical benefit this design does deliver comes from runtime selection of a shallower *fixed* iteration depth (18/14/11 cycles), which reduces cycles without touching F_{max} at all – unlike either acceleration mechanism. We report this negative result for both mechanisms deliberately and in full alongside the positive adaptive-precision result, because a hardware paper's cycle-count claims are only as credible as the timing analysis behind them; we also note, respectfully, that [27]'s own reported throughput figures appear to assume zero F_{max} penalty for the same class of mechanisms (Section 5.4), an assumption our post-synthesis results do not support in our flow. The design retains a fully multiplier-free, shift-add datapath throughout, requires no repeated-index correction for either the rotation or vectoring stage, and all RTL, testbenches, synthesis scripts, and raw results (including the netlists analyzed by OpenSTA) are provided so that the throughput reversal in Table 15 can be independently reproduced or challenged. Future work includes a pipelined DSPC variant that could recover real throughput (Section 5.1), extending the adaptive-precision criterion to hyperbolic-mode functions, and a process/toolchain-matched comparison against commercial FPGA CORDIC IP and against [27]'s own architecture once retargeted to a common technology – closing the normalization gap that Section 5.5's vendor-datasheet comparison still carries.

Data and code availability. The synthesizable Verilog (`cordic_engine.v`, `cordic_top.v`), testbenches, the Python reference model and precision-mode sweep scripts, the Yosys/OpenSTA synthesis and timing scripts, the SKY130 and iCE40 synthesis outputs, and the raw numerical results underlying every table and figure in this paper are available in a public repository, with the link and/or DOI to be finalized at submission.

References

[1] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959.

[2] J. S. Walther, "A unified algorithm for elementary functions," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1971, pp. 379–385.

[3] H. M. Ahmed, J.-M. Delosme, and M. Morf, "Highly concurrent computing structures for matrix arithmetic and signal processing," *IEEE Computer*, vol. 15, no. 1, pp. 65–82, Jan. 1982.

[4] R. Andraka, "A survey of CORDIC algorithms for FPGA based computers," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA)*, Monterey, CA, USA, 1998, pp. 191–200.

[5] Y. Luo, Y. Wang, H. Sun, Y. Zha, Z. Wang, and H. Pan, "CORDIC-based architecture for computing Nth-root and its implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 12, pp. 4183–4195, Dec. 2018.

[6] A. Changela, Y. Kumar, M. Woźniak, J. Shafi, and M. F. Ijaz, "Radix-4 CORDIC algorithm based low-latency and hardware efficient VLSI architecture for Nth root and Nth power computations," *Sci. Rep.*, vol. 13, art. no. 20918, 2023.

[7] M. Qin, T. Liu, B. Hou, Y. Gao, Y. Yao, and H. Sun, "A low-latency RDP-CORDIC algorithm for real-time signal processing of edge computing devices in smart grid cyber-physical systems," *Sensors*, vol. 22, art. no. 7489, 2022.

[8] F. Salehi *et al.*, "Novel design for a low-latency CORDIC algorithm for sine-cosine computation and its implementation on FPGA," *Microprocessors and Microsystems*, 2020.

[9] W. Shen, J. Jiang, M. Li, and S. Liu, "Efficient CORDIC-based activation functions for RNN acceleration on FPGAs," *IEEE Trans. Artif. Intell.*, 2024.

[10] N. A. Mohamed and J. R. Cavallaro, "A unified parallel CORDIC-based hardware architecture for LSTM network acceleration," *IEEE Trans. Comput.*, vol. 72, pp. 2752–2766, Oct. 2023.

[11] P. Poczekajlo *et al.*, "Low latency recoding CORDIC algorithm for FPGA implementation," in *Proc. Int. Conf. Comput. Sci. (ICCS)*, 2025.

[12] C.-H. Lin and A.-Y. Wu, "Mixed-scaling-rotation CORDIC (MSR-CORDIC) algorithm and architecture for high-performance vector rotational DSP applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 52, no. 11, pp. 2385–2396, Nov. 2005.

[13] Y. Luo, Y. Wang, Y. Ha, Z. Wang, S. Chen, and H. Pan, "Generalized hyperbolic CORDIC and its logarithmic and exponential computation with arbitrary fixed base," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 9, pp. 2156–2169, Sep. 2019.

[14] Y. Liu, L. Fan, and T. Ma, "A modified CORDIC FPGA implementation for wave generation," *Circuits Syst. Signal Process.*, vol. 33, pp. 321–329, 2014.

[15] A. Aggarwal *et al.*, "Redesigned-scale-free CORDIC algorithm based FPGA implementation of window functions to minimize area and latency," *Int. J. Reconfigurable Comput.*, vol. 2012, art. no. 185784, 2012.

[16] A. G. M. Strollo, "Digital synthesizer/mixer with hybrid CORDIC-multiplier architecture: error analysis and optimization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 56, no. 2, pp. 364–373, Feb. 2009.

[17] A. Changela *et al.*, "Mixed-radix, virtually scaling-free CORDIC algorithm based rotator for DSP applications," *Integration, the VLSI Journal*, 2021.

[18] C. Wolf, "Yosys – a free Verilog synthesis suite," in *Proc. 21st Austrian Workshop Microelectron. (Austrochip)*, 2013.

[19] M. Shalan and T. Edwards, "Building OpenLane: A 130nm OpenROAD-based tapeout-proven flow," in *Proc. 39th Int. Conf. Comput.-Aided Design (ICCAD)*, 2020, pp. 1–6.

[20] T. Ajayi *et al.*, "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain," in *Proc. GOMACTECH*, 2019.

[21] F.-G. Stoica, A. Calinescu, and M. Enachescu, "ROM-less co(sine) synthesizer," *Electronics*, vol. 15, no. 5, art. no. 1093, 2026.

[22] Q. K. Omran, M. T. Islam, N. Misran, and M. R. I. Faruque, "A ROM-less direct digital frequency synthesizer based on hybrid polynomial approximation," *Int. J. Antennas Propag.*, vol. 2014, art. no. 812576, 2014.

[23] S. H. Ibrahim, S. H. Md. Ali, and M. S. Islam, "Hardware implementation of 32-bit high-speed direct digital frequency synthesizer," *The Scientific World Journal*, vol. 2014, art. no. 131568, 2014.

[24] J. Cherry (Parallax Software, Inc.), "OpenSTA: Parallax static timing analyzer," *GitHub repository*, `https://github.com/parallaxsw/OpenSTA`, accessed 2026.

[25] AMD/Xilinx, "CORDIC v6.0 LogiCORE IP Product Guide," PG105, Aug. 2021.

[26] AMD/Xilinx, "LogiCORE IP CORDIC v5.0 Data Sheet (AXI)," DS858, Oct. 2011.

[27] J. Zhang, Z. Chen, and C. Li, "Design and implementation of an adaptive-precision high-throughput CORDIC-based Nth-root architecture," *SSRN preprint*, abstract no. 6434702, 2026. Preprint, not peer reviewed. `https://ssrn.com/abstract=6434702`.